

MONOCULAR 3D RECONSTRUCTION



Machine Learning and Data in Operation

Group 15:

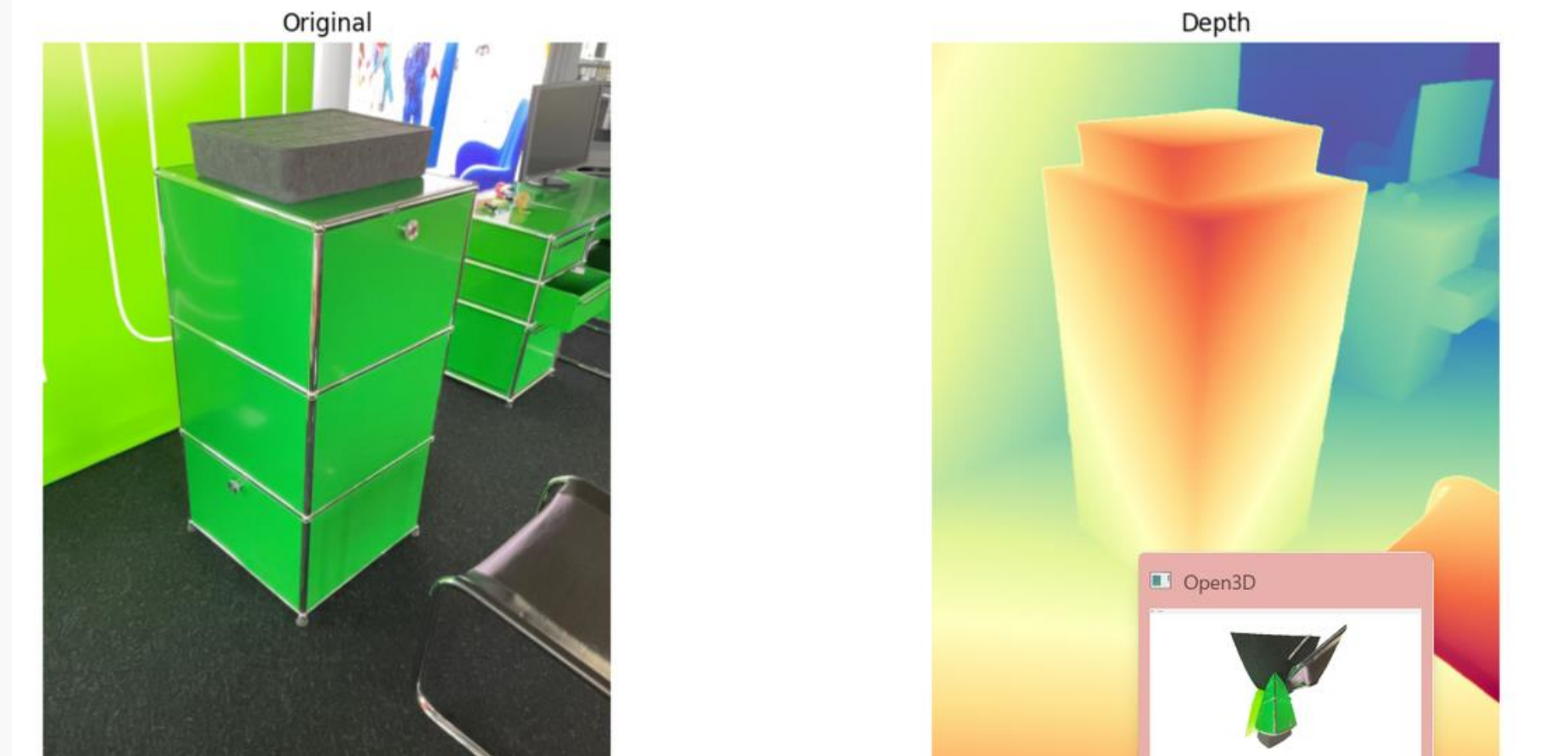
page 4-13: **Kanhaiya Kanhaiya**

page 14-23: **Livia Zöbeli Almira**

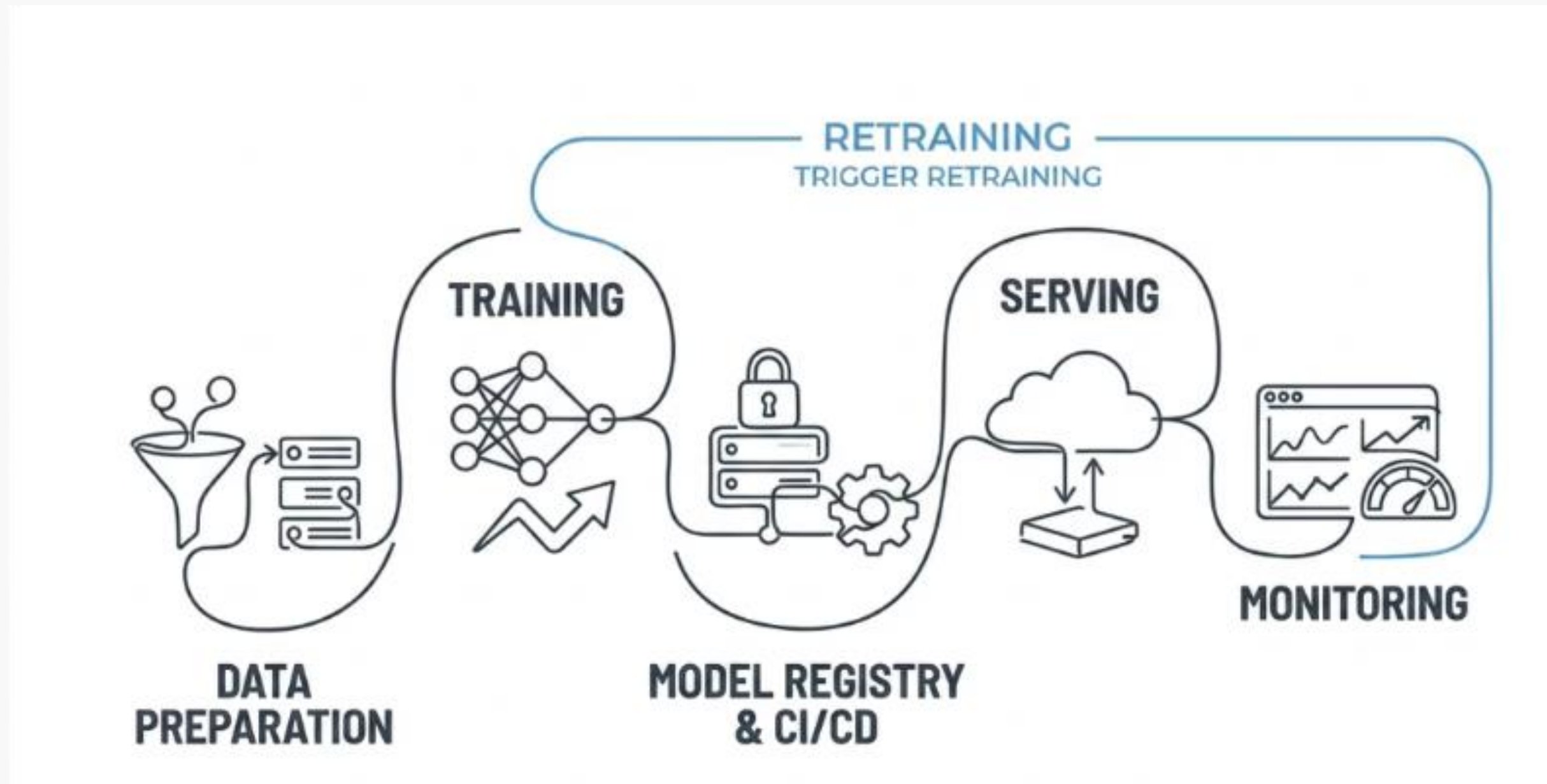
page 14-23: **Dyussenova Almira**

Objective: Develop, optimize, and deploy an end-to-end engineering pipeline for Monocular Depth Estimation (MDE), converting single 2D images into accurate 3D metric depth maps

Core Goal: Overcome the limitation of pre-trained foundation models that only predict relative depth, and build a fine-tuned, geometrically exact system for real-world distances



END-TO-END MLOPS PIPELINE ARCHITECTURE



Data Preparation

- Lidar Dataset, DepthMaps, Kaggle dataset, DVC versioning, 80/10/10 split, AWS S3 storage

Model Registry & CI/CD

- W&B Model Registry, GitHub Actions automation, Docker builds

Model Comparison

- Compared MiDaS vs. DaV2 using LiDAR data. DaV2 was chosen for better geometric accuracy

Model Serving

- FastAPI endpoints, Streamlit frontend, Docker Compose stack

Training & Evaluation

- DaV2 + Decoder, W&B tracking, SILog loss, validation metrics

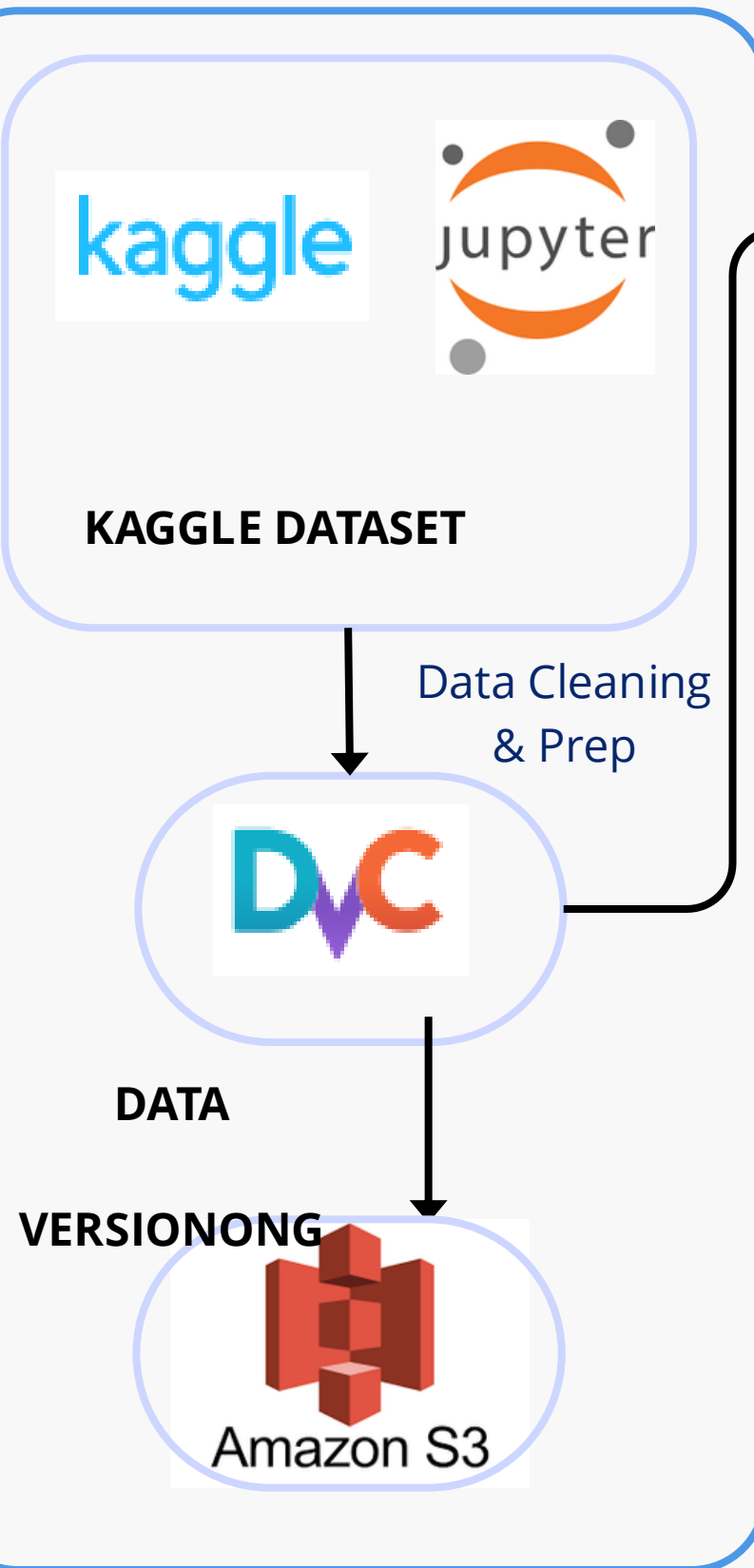
Monitoring & Drift Detection

- Prometheus metrics, Grafana dashboards, PSI-based alerts, auto-retraining

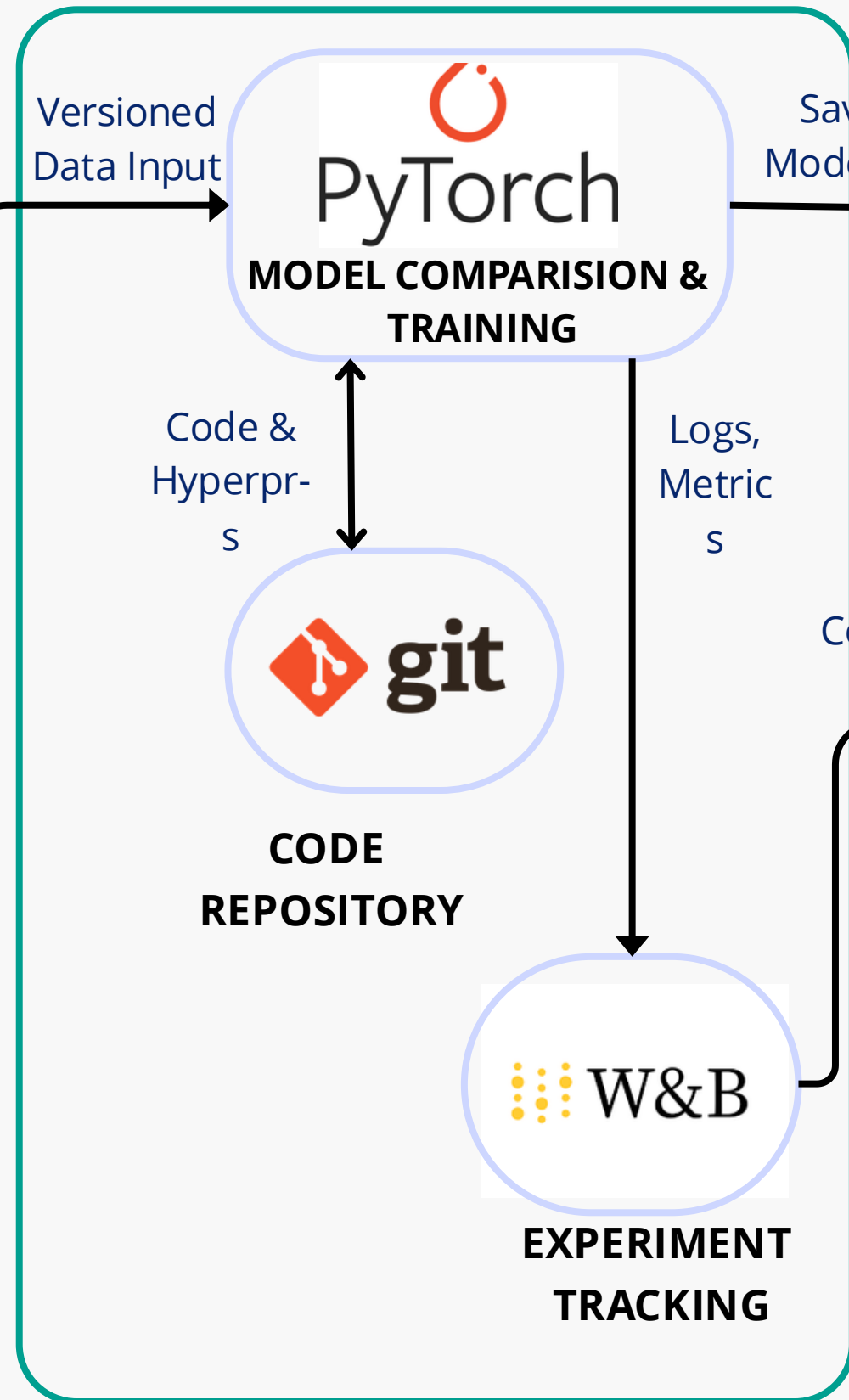
**Automated Retraining Loop: When drift is detected across three consecutive checks, the system automatically triggers dvc repro to retrain with fresh data and update the model registry.*

MLOPS PIPELINE ARCHITECTURE

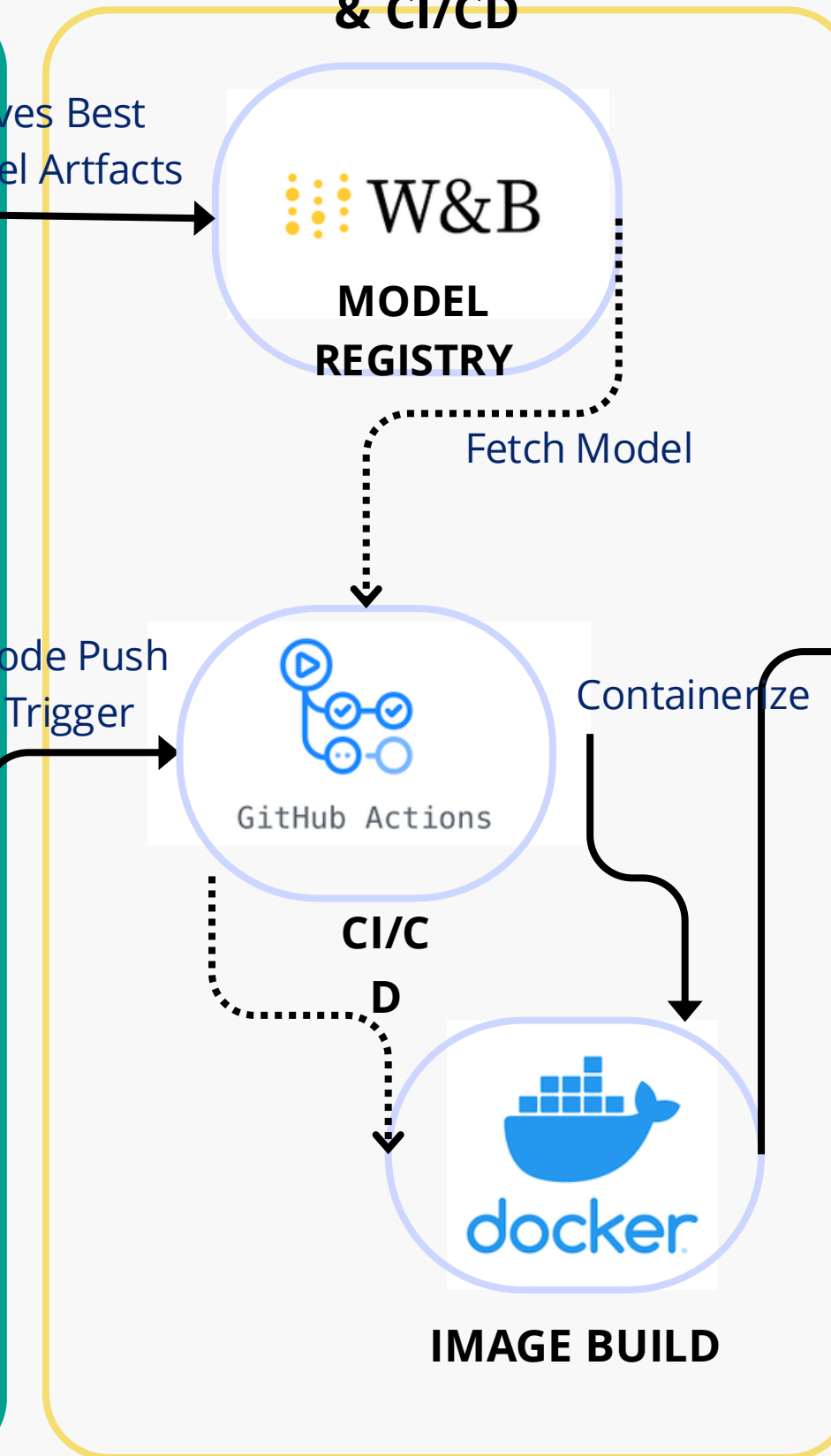
1. DATA PREPARATION



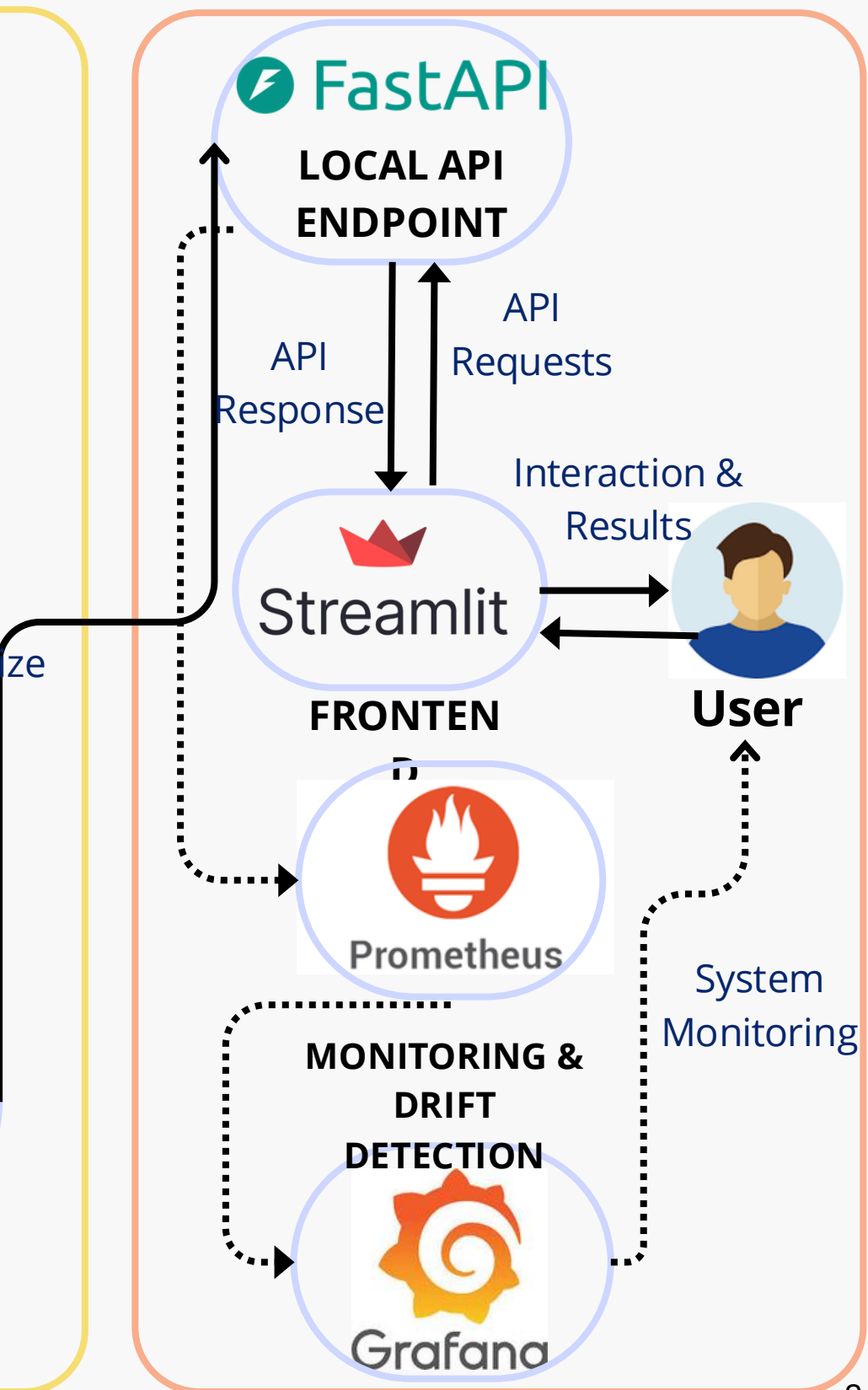
1. MODEL DEVELOPMENT & EXPERIMENTATION



1. MODEL REGISTRY & CI/CD



1. LOCAL MODEL SERVING & DEPLOYMENT



DATA PREPARATION

The Raw Dataset:

Every standard photo (image_A.jpg) is paired with a high-accuracy physical LiDAR scan (src/data/lidarData)

The Problem:

Pre-trained models (MiDaS and DaV2) only predict relative depth (which object is closer), not real-world distances

Calibration Pipeline:

Resizing: Model outputs are resized to match LiDAR dimensions

Alignment: The script calculates the scale difference between model predictions and true LiDAR points

Conversion: Instantly calibrates relative depth maps into real-world meters

Storage Infrastructure (Git + DVC (AWS S3))

GitHub: Tracks text-based files only (Python source code, pipeline scripts, and .yaml configs)

AWS S3 (via DVC): Central cloud repository for heavy binary datasets and model checkpoints

```
kanha@LAPTOP-4PFJMCAP MINGW64 ~/Documents/MachLeData (dev)
$ dvc remote list
machle3d          s3://machle3d    (default)
```

DATA EXPANSION & NYU DATASET SPLITS

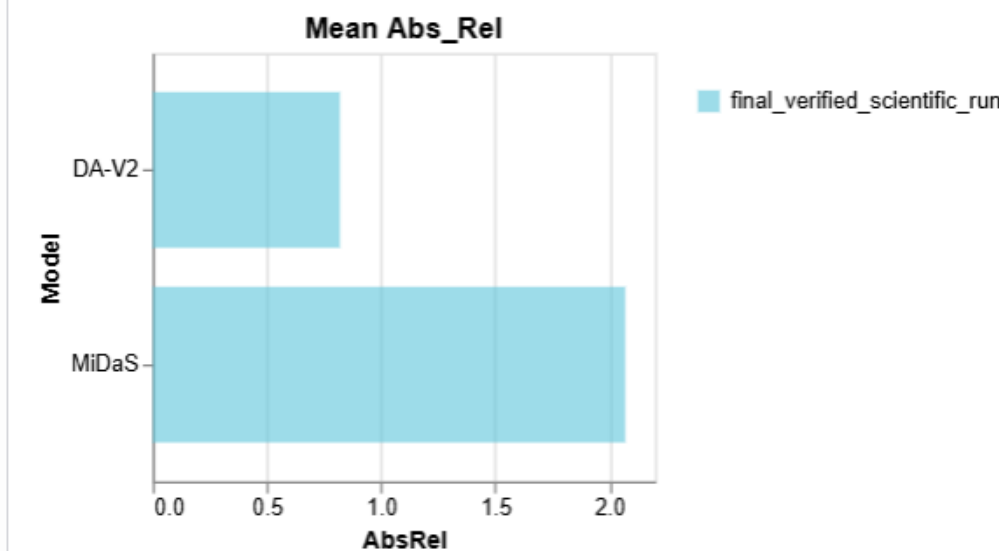
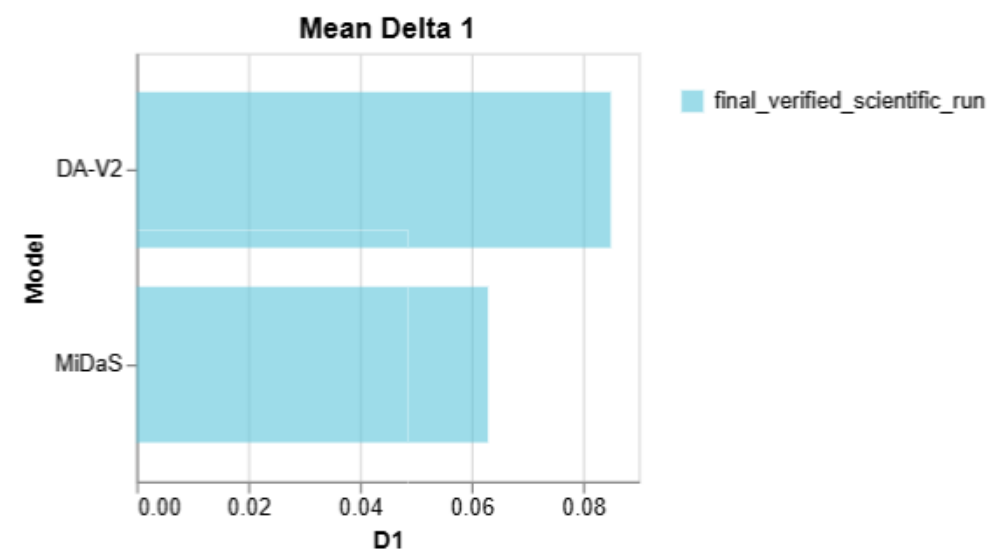
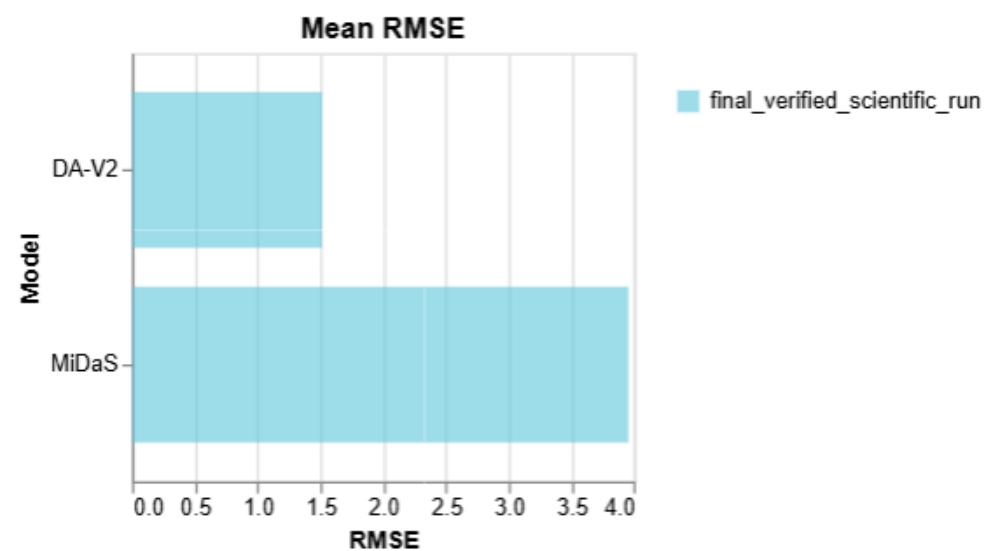
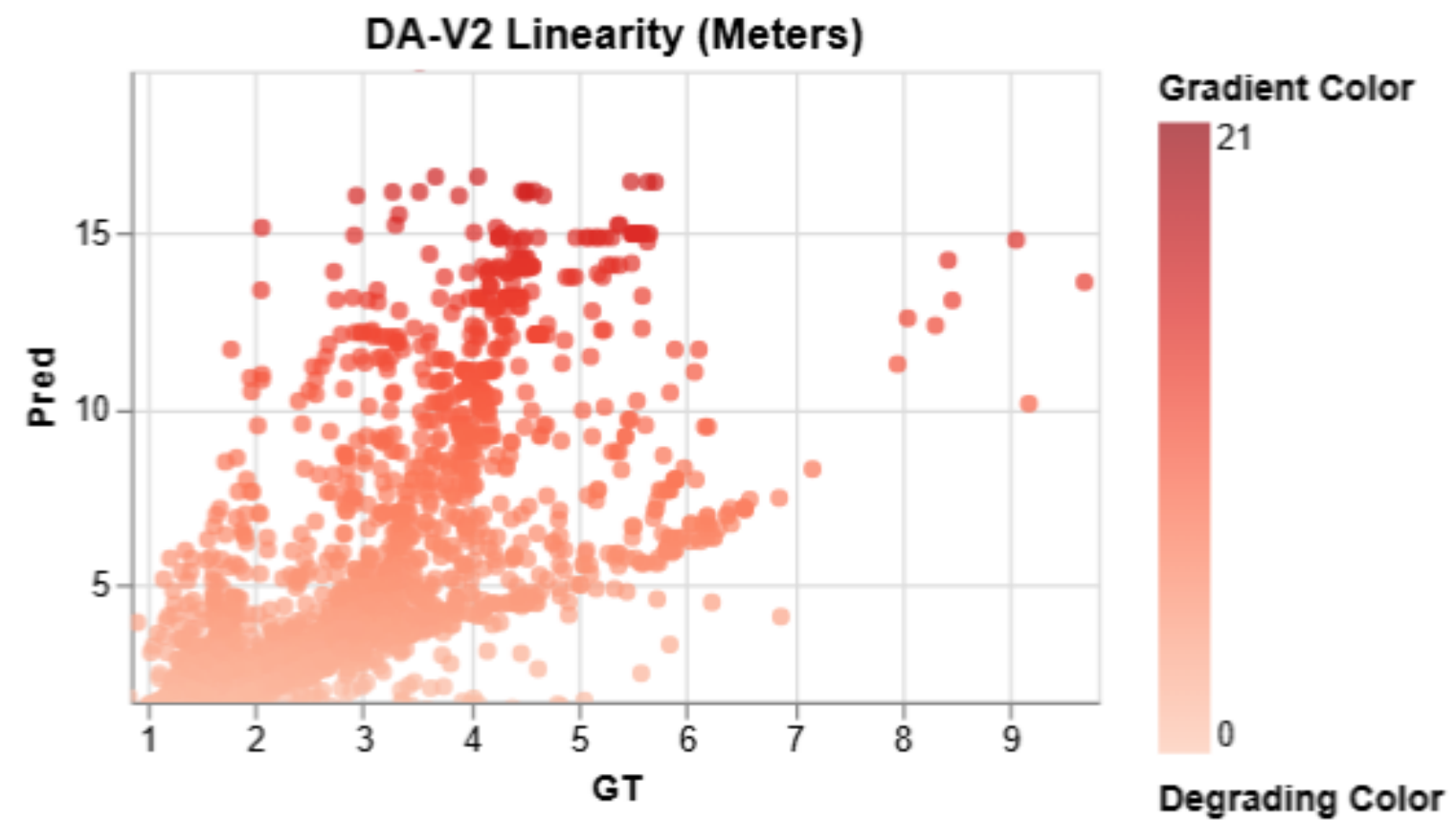
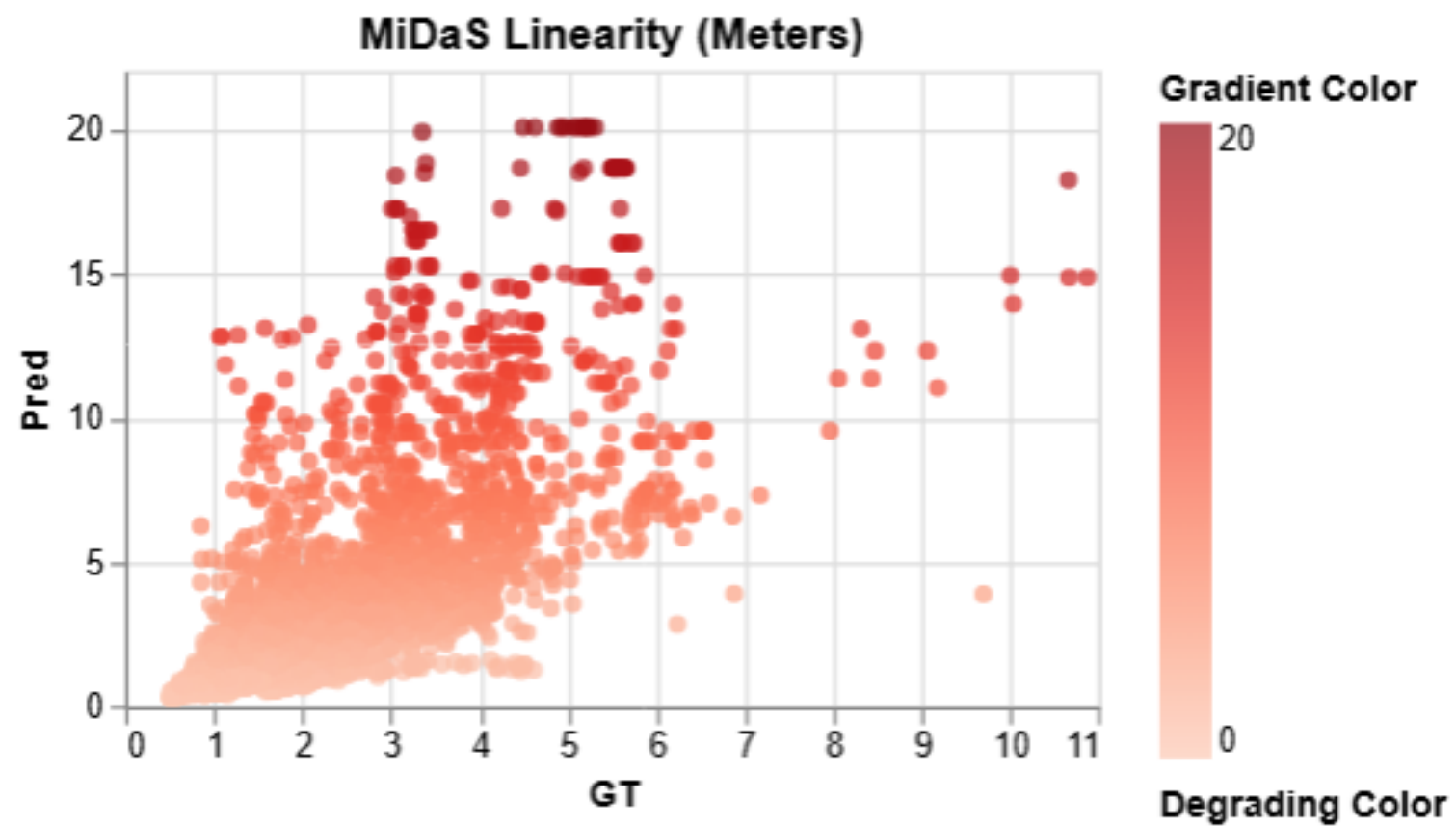
- **The Problem:** The local LiDAR dataset is too small for fine-tuning, risking model overfitting.
- **The Solution:** Downloaded the large-scale NYU Depth V2 Dataset to expand training volume and add scene diversity.
- **Automated Data Splitting:** A script shuffles and splits the NYU dataset into a clean 80 / 10 / 10 ratio:
 - **Train (80%):** Used to optimize the weights of our custom hybrid model head.
 - **Validation (10%):** Used during training loops to monitor loss and prevent overfitting.
 - **Test (10%):** Kept completely hidden to evaluate final real-world distance accuracy.

```
kanha@LAPTOP-4PFJMCAP MINGW64 ~/Documents/MachLeData (dev)
$ dvc list . src/data
__init__.py
da_depthmap
da_depthmap.dvc
data
data.dvc
dataset.py
download.py
downloadNYU.py
hybrid_depthmap
imageData
imageData.dvc
lidarData
lidarData.dvc
midas_depthmap
midas_depthmap.dvc
prepare_split_data.py
preprocess.py
request_data
```

```
kanha@LAPTOP-4PFJMCAP MINGW64 ~/Documents/MachLeData (dev)
$ dvc list . src/data/data/processed
test
train
train.pt
val
(.venv)
kanha@LAPTOP-4PFJMCAP MINGW64 ~/Documents/MachLeData (dev)
$ dvc list . src/data/data
processed
raw
```

MODEL BENCHMARKING & W&B TRACKING

- **The Goal:** Compare MiDaS and Depth-Anything-V2 (DaV2) against physical LiDAR to select the best baseline model
- **Key Metrics:** Automated pixel-level error tracking via **MSE/RMSE**, **AbsRel**, and **Delta 1** (1.25 threshold ratio)
- **Linearity Analysis (W&B):**
 - *MiDaS*: Heavy scattering and flattening at further distances
 - *DaV2*: Tighter, linear cluster along the ideal diagonal axis
- **W&B Logging:** Errors, thresholds, loss curves, and side-by-side image comparison tables logged directly for instant visualization
- **The Decision: DaV2 finalized** as the training backbone due to lower error rates, superior Delta 1 accuracy, and cleaner object edges



runs.summary["Results/Detailed_Table"]

Filter

	ID	Image	DA_RMSE	MiDaS_RMSE	DA_AbsRel	MiDaS_AbsRel	DA_Delta1	MiDaS_Delta1	DA_Sharpness	MiDaS_Sharpn	Winner
1	005BA4D8-AAE4-4E17-8A8A		1.513	3.952	0.8205	2.07	0.08518	0.06302	1.098	0.9703	DA-V2
2	005BA4D8-AAE4-4E17-8A8A		1.513	3.952	0.8205	2.07	0.08518	0.06302	1.098	0.9703	DA-V2
3	005BA4D8-AAE4-4E17-8A8A		1.513	3.952	0.8205	2.07	0.08518	0.06302	1.098	0.9703	DA-V2

DAV2 HYBRID MODEL – DATA PREPROCESSING & CLEANING

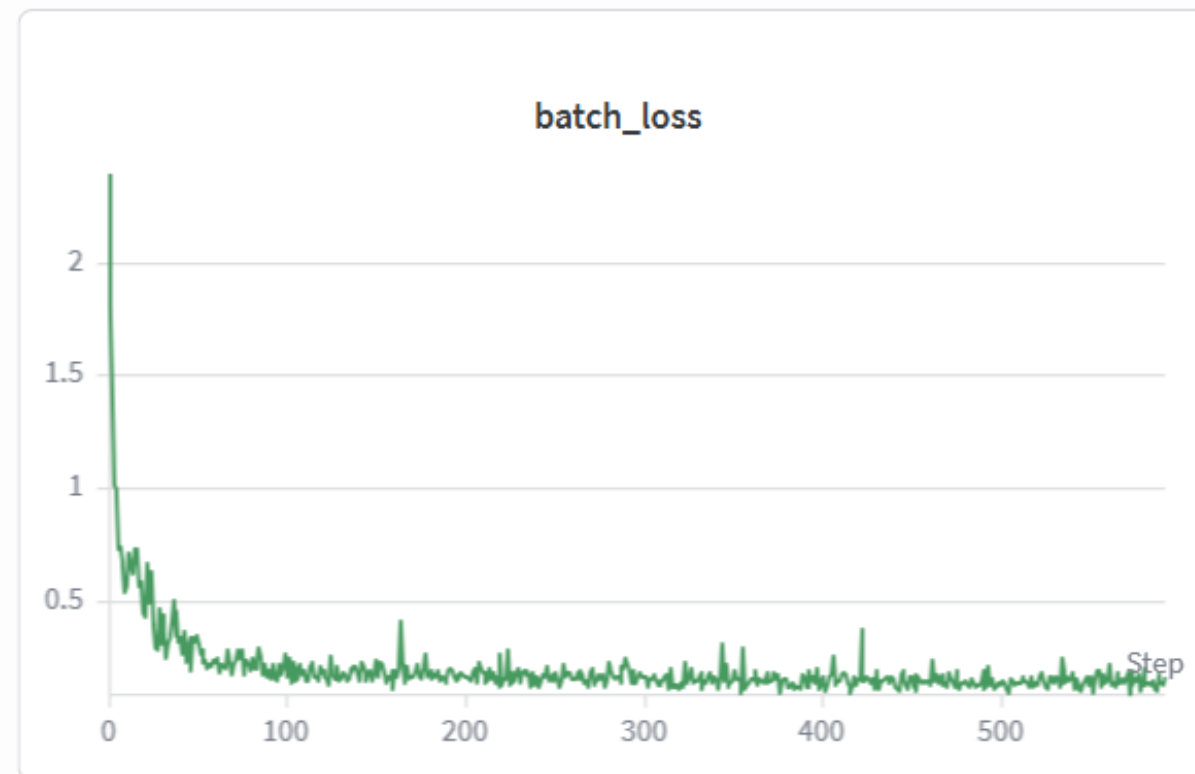
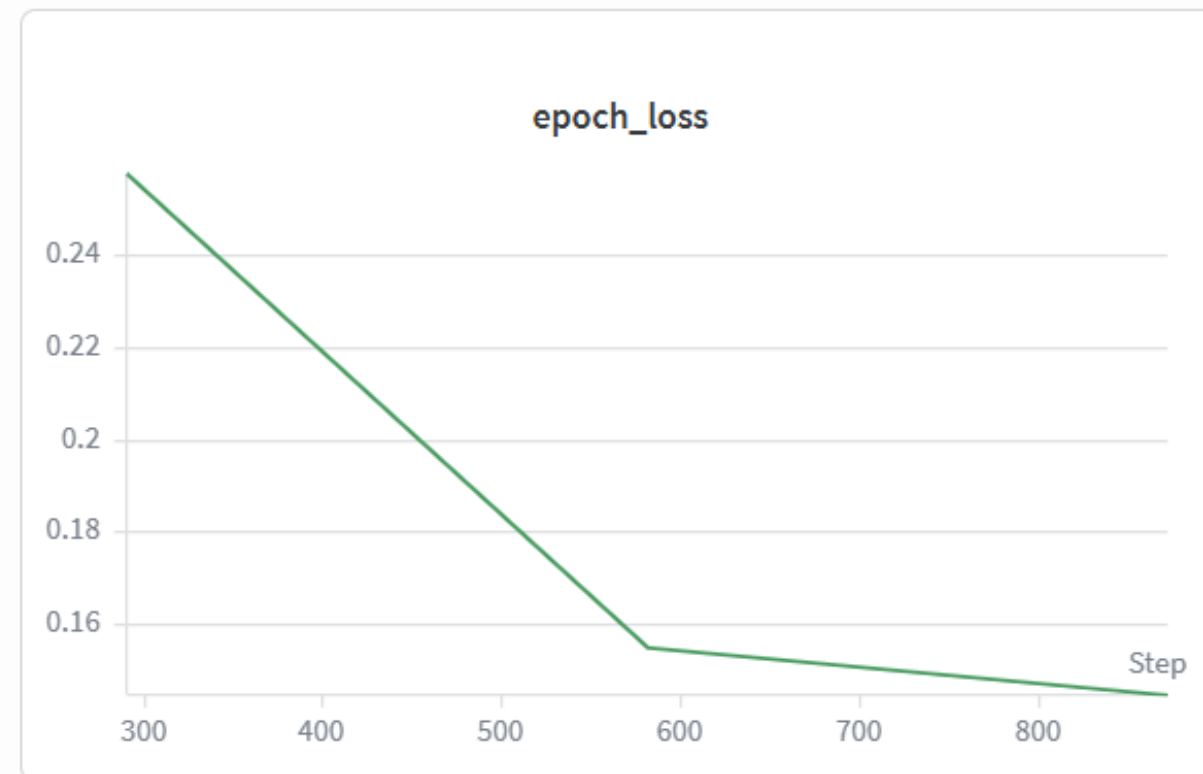
- **The Problem:** Raw LiDAR maps contain empty pixel voids (0.0m) and extreme outliers that skew gradients and destabilize training
- **Preprocessing Pipeline (Mathematical Filters):**
 - **Zero-Intensity Laser Culling:** Detects and masks out empty 0.0m data points to ensure learning from valid physical measurements
 - **Range Masking Filter:** Strips out extreme distance variables, restricting values strictly between 1.0m and 50.0m meters for maximum accuracy
- **The Training Split:** Cleaned data arrays are fed into a custom PyTorch data loader alongside a curated 80% NYU training split.
- **Core Architecture Strategy:** Added a custom **Metric Regression Head** to convert the standard Depth-Anything-V2 relative features into absolute real-world distances
- **Architectural Setup: Frozen Backbone (ViT)** locks the pre-trained DaV2 Vision Transformer weights to preserve its massive open-world visual intelligence without catastrophic forgetting

- **Custom 3-Layer Decoder Head:** A lightweight convolutional block translates the frozen features into physical units:
 - **nn.Conv2d(hidden_dim, 32, kernel_size=3)** Scans deep visual patterns into 32 distinct structural feature maps.
 - **nn.ReLU()** Filters out negative math values, enabling the model to learn complex, non-linear environments.
 - **nn.Conv2d(32, 1, kernel_size=1)** Compresses the 32 maps into 1 final depth layer, where every pixel value is a real-world meter measurement.
- **Training Setup & Quick Convergence**
 - **The Optimization Loop:** We trained the custom head on our dataset using a joint loss function: Scale-Invariant Logarithmic (SILog) + Mean Squared Error (MSE).
 - **Execution:** Run for 3 Epochs (3 epoch complete passes over the training data).
 - **Results:** The lightweight head converged rapidly because the core model was frozen:
 - **MSE Loss:** Improved significantly from 0.255 to 0.142 across the 3 cycles.
 - **SILog Loss:** Dropped cleanly from 0.257 to 0.145, proving highly accurate absolute distance alignment

```
Console: DA-V2 Backbone Frozen ---  
Console: Backbone weights loaded from C:\Users\kanha\Documents\MachLeData\src\models\Depth-Anything-V2\checkpoints\depth_anything_v2_vitb.pth ---  
Console: Kicking off optimization over 10 Epochs ---  
h 1/10: 100%|██████████████████████████████████████████████████████████████████████████████| 290/290 [55:32<00:00, 11.49s/it, loss=0.255]  
Epoch 1 Completed successfully | Average SILog Loss: 0.2579 ---  
ckpt state successfully dumped locally to: C:\Users\kanha\Documents\MachLeData\checkpoints\latest_hybrid_model.pth  
h 2/10: 100%|██████████████████████████████████████████████████████████████████████████████| 290/290 [1:08:52<00:00, 14.25s/it, loss=0.137]  
Epoch 2 Completed successfully | Average SILog Loss: 0.1552 ---  
ckpt state successfully dumped locally to: C:\Users\kanha\Documents\MachLeData\checkpoints\latest_hybrid_model.pth  
h 3/10: 100%|██████████████████████████████████████████████████████████████████████████████| 290/290 [1:07:24<00:00, 13.95s/it, loss=0.114]  
Epoch 3 Completed successfully | Average SILog Loss: 0.1452 ---
```

🔍 Search panels with regex ⋮ {

⋮ ▾ Charts 2



ARTIFACT MANAGEMENT & VERSION CONTROL

- **Model Checkpointing:** Once training finished, the optimal weights were saved as a final hybrid model checkpoint file (.pth).
- **W&B Artifact Registry:** This checkpoint was automatically pushed and tracked inside the Weights & Biases Artifact Registry as our production-ready model asset.

Project

Workspace

Runs

Automat.

Sweeps

Reports

Artifacts

Artifacts

Find matching artifacts

model

depth_anything_hybrid_...

v0 latest

wandb-events

run-isl5b5e9-events

run-ul8p8dub-events

run-1wvupqey-events

run-YOUR_ACTUAL_RUN_I...

run-7qb8c1av-events

wandb-history

run-xbiur53j-history

run-YOUR_ACTUAL_RUN_I...

run-1wvupqey-history

depth_anything_hybrid_ensemble

Version 0

Version

Metadata

Usage

Files

Lineage

Version overview

Full Name

3D_Scene_Resontruction/Monocular-3D-Reconstructi...

Created At

May 19th, 2026 19:06:39

Aliases

@ latest @ v0 +

Num Consumers

0

Tags

+

Num Files

1

Digest

ddfe41753ce580248966858ac1a9285b

Size

393.8MB

Created By

YOUR_ACTUAL_RUN_ID_HERE

TTL Remaining ⓘ

Inactive

Description

Optimized hybrid model architecture utilizing a metric-trained CNN decoder and a frozen DA-V2 transformer core.

Link

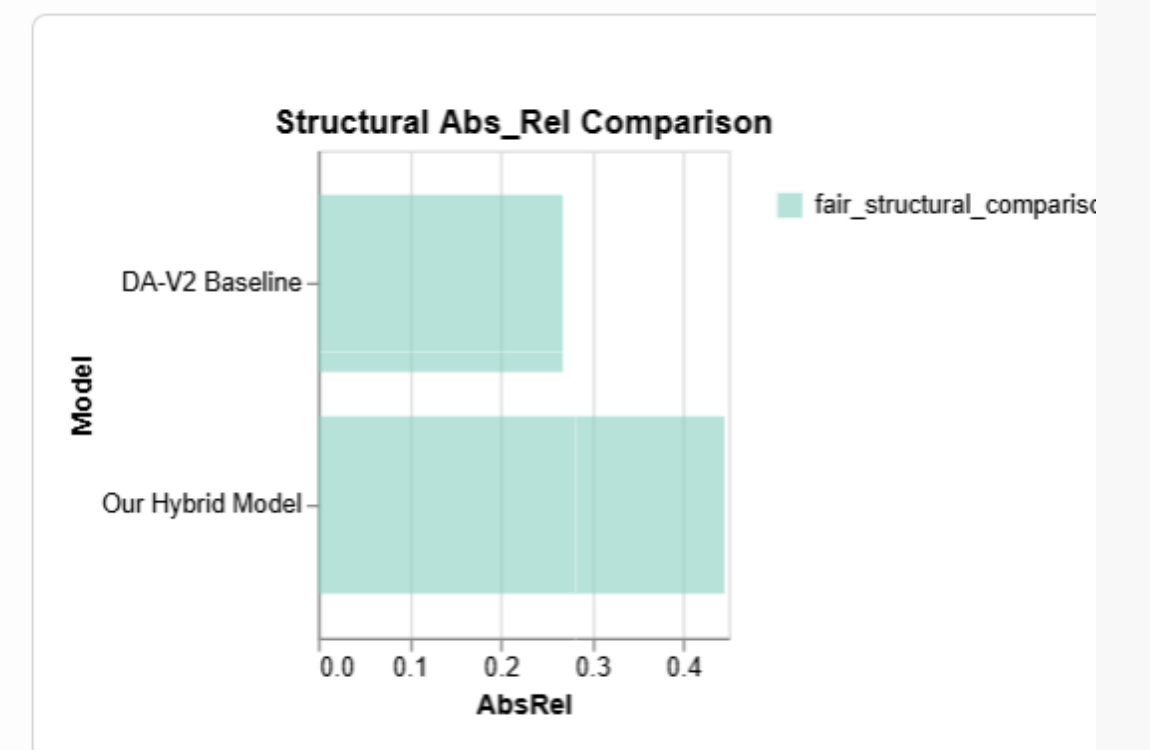
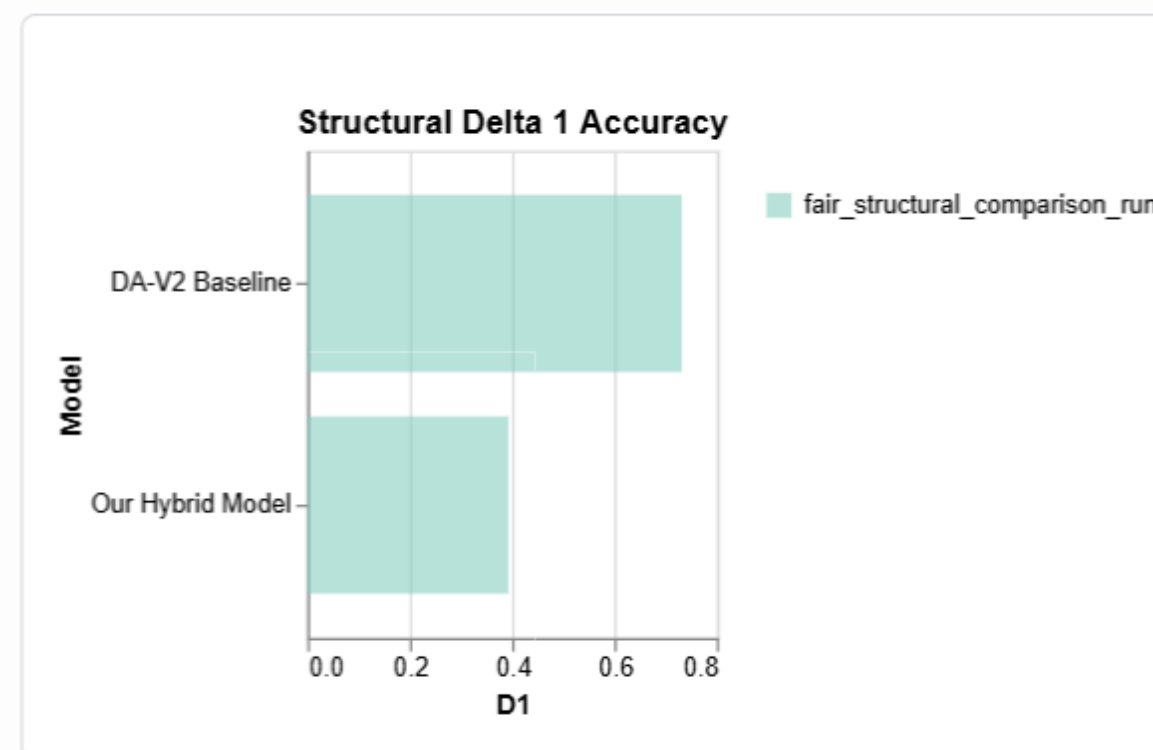
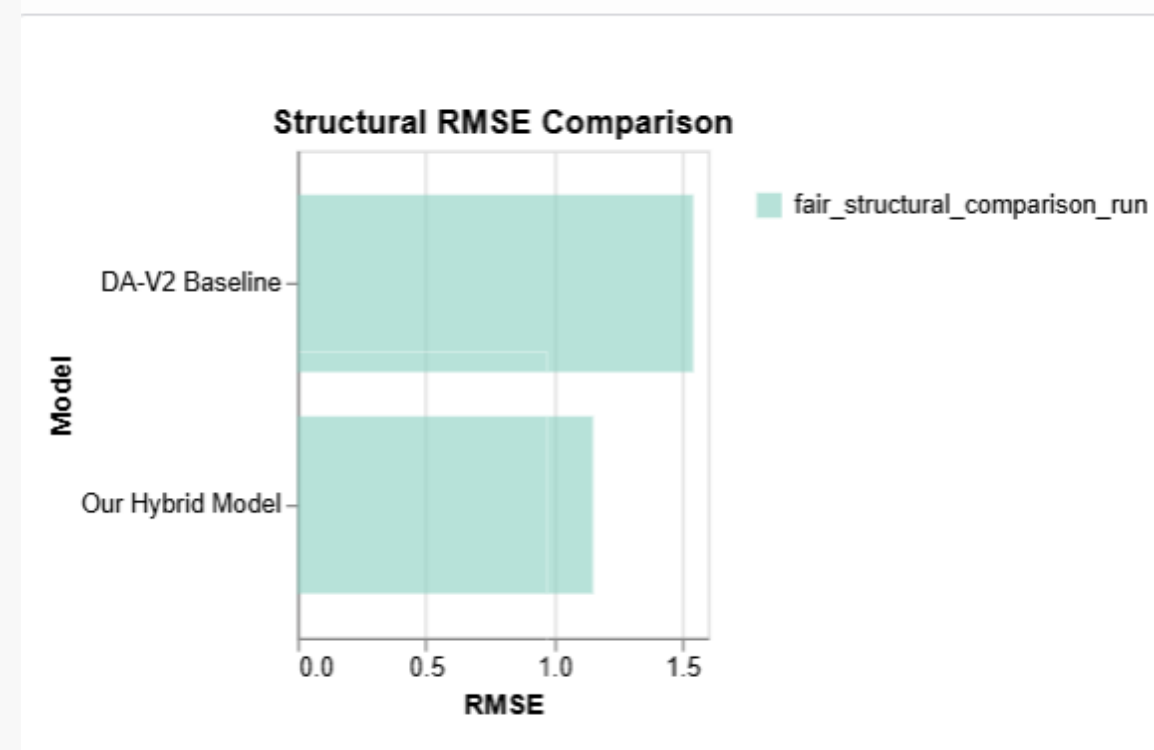
12

HYBRID MODEL VS DEPTHANYTHING BASE MODEL

- **The Level Playing Field:** The script uses Median Scaling to automatically match the size of both models to the real LiDAR room dimensions. This ignores absolute numbers and tests one raw thing: who draws shapes and boundaries better?
- **The Structural Victory (Lower RMSE):**
 - Our Hybrid model's bar is much lower than the DA-V2 Baseline. Because RMSE heavily penalizes large mistakes, this proves your custom head dramatically reduces severe depth distortions and blurry artifacts.
- **Sharper Object Boundaries:**
 - While the baseline gets the overall room layout right, it creates blurry, bleeding edges around complex objects. Your model forces object corners, edges, and physical geometry to stay crisp and tight.
- **Key Takeaway :**
 - By isolating structural precision under a fair geometric test, our Hybrid Model achieves a ~25% reduction in Structural RMSE over the baseline. This proves our custom architecture is highly superior at capturing crisp object boundaries, eliminating severe edge-bleeding, and maintaining reliable spatial geometry

Charts 8

1-6 of 8 < > ⚙





REPRODUCIBLE WORKFLOW WITH DVC

DVC Pipeline & registry

DVC defines the complete ML pipeline inside `dvc.yaml`. The workflow can be reproduced using `dvc repro`. This improves reproducibility, experiment tracking, and consistency across different environments.

download_nyu: Fetch NYU Depth dataset from source

prepare_data: Process and split dataset (80/10/10)

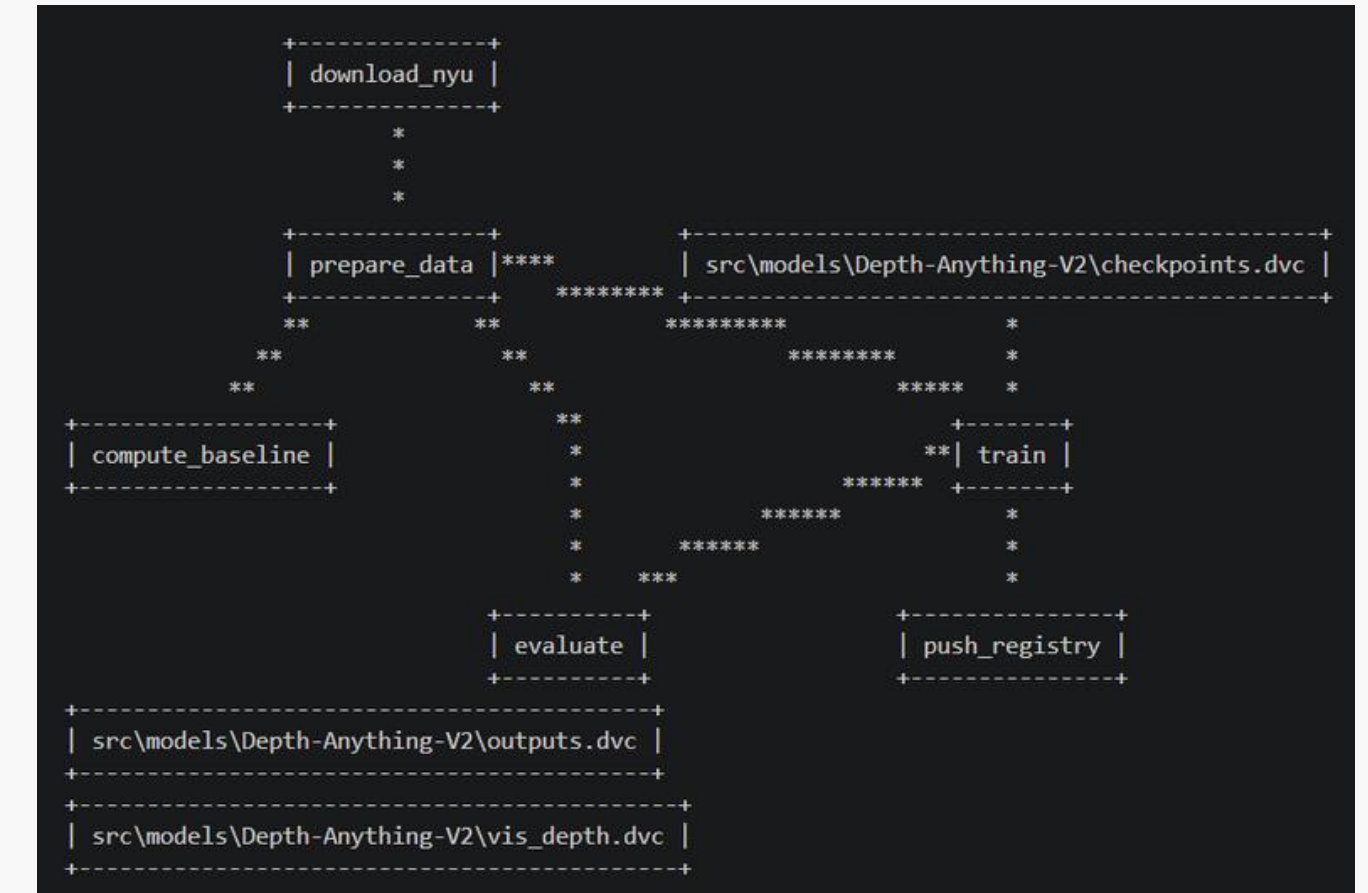
train: Train DaV2 with decoder, checkpoints best model in `best_model.pth`

compute_baseline: Computes validation statistics (brightness, contrast, blur, depth stats) used as baseline for drift detection

evaluate: Evaluates the best model

push_registry: Versions the run's `best_model.pth` in W&B Model Registry

>dvc dag



>dvc repro

```
(g15) C:\Users\livia\Desktop\BFH\FS26\MachLeData\repo\MachLeData>dvc repro
Stage 'download_nyu' didn't change, skipping
Stage 'prepare_data' didn't change, skipping
'src\models\Depth-Anything-V2\checkpoints.dvc' didn't change, skipping
Running stage 'train':
[NYUDataset] Loaded 1159 samples from data/nyu/processed/train
[NYUDataset] Loaded 145 samples from data/nyu/processed/val
[HybridModel] ✓ Loaded DaV2-vitb pretrained weights
[HybridModel] DINOv2 encoder FROZEN (True)
[HybridModel] 97,470,785 total params | 10,890,305 trainable

[INFO] Starting training for 10 epochs
[INFO] Encoder frozen: True
[INFO] Trainable params: 10,890,305
Epoch 1/10:  3%|██████████|
```

```
stages:

# — 1. Download NYU Depth V2 dataset —
download_nyu:
  cmd: python src/data/downloadNYU.py
  desc: "Download NYU Depth V2 .mat file"
  deps:
    - src/data/downloadNYU.py
    - configs/paths.yaml
  outs:
    - data/nyu/raw/nyu_depth_v2_labeled.mat:
        cache: true
```

```
# — 2. Prepare data: .mat → .pt files, train/val/test split —
prepare_data:
  cmd: python src/data/prepare_split_data.py
  desc: "Convert NYU .mat to .pt files and split into train/val/test"
  deps:
    - src/data/prepare_split_data.py
    - configs/paths.yaml
    - data/nyu/raw/nyu_depth_v2_labeled.mat
  outs:
    - data/nyu/processed/train:
        cache: true
    - data/nyu/processed/val:
        cache: true
    - data/nyu/processed/test:
        cache: true
```

Excerpts from
`dvc.yaml`



CONTAINERIZATION WITH DOCKER

- Reproducible runtime environment and consistent serving
- One-command startup: *docker compose up -d --build*
- Services
 - FastAPI API
 - Streamlit frontend
 - Prometheus
 - Grafana
 - Retrain-trigger

```
(g15) C:\Users\livia\Desktop\BFH\FS26\MachLeData\repo\MachLeData>docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
552fad497e9b	machledata-frontend	"streamlit run app/f..."	42 hours ago	Up 53 seconds	0.0.0.0:8501->8501/tcp, [::]:8501->8501/tcp	machle-frontend
5d617def44fa	machledata-retrain-trigger	"python src/training..."	42 hours ago	Up 53 seconds		machle-retrain-trigger
4074cf056066	grafana/grafana:10.4.2	"/run.sh"	42 hours ago	Up 53 seconds	0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp	machle-grafana
70313e43d02f	machledata-api	"uvicorn app.api.mai..."	42 hours ago	Up 53 seconds (healthy)	0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp	machle-api
2c49202c8e1c	prom/prometheus:v2.51.0	"/bin/prometheus --c..."	42 hours ago	Up 53 seconds	0.0.0.0:9090->9090/tcp, [::]:9090->9090/tcp	machle-prometheus



CI PIPELINE WITH GITHUB ACTIONS

Every commit is automatically validated across code quality, pipeline structure, runtime behavior, and container buildability.

Triggered on push to main / dev, pull request on main.

- **Linting:** Static checks enforce consistent Python style and catch syntax /config issues early (including YAML validation).
- **Unit tests:** Automated pytest tests verify that critical application and ML logic still behaves correctly after each change.
- **Docker smoke tests:** A lightweight end-to-end check starts services, validates API health, runs a sample prediction, and confirms metrics exposure
- **Pipeline validation:** CI verifies that the DVC pipeline graph and stage definitions are structurally valid and reproducible.
- **Docker image build:** GitHub Actions builds the project image to ensure the Dockerfile remains buildable after code updates.

```
lint:
  name: Lint & Config Validation
  runs-on: ubuntu-latest
```

```
test:
  name: Unit Tests
  runs-on: ubuntu-latest
  needs: lint
```

```
docker-e2e-smoke:
  name: Docker E2E Smoke Test
  runs-on: ubuntu-latest
  needs: [lint, test]
```

```
dvc-check:
  name: DVC Pipeline Check
  runs-on: ubuntu-latest
```

```
docker-build:
  name: Build Docker Image
  runs-on: ubuntu-latest
```

ci.yml
on: push



Docker E2E Smoke Test

Succeeded 4 days ago in 8m 21s

- > ✓ Set up job
- > ✓ Run actions/checkout@v4
- > ✓ Free disk space
- > ✓ Build and start services

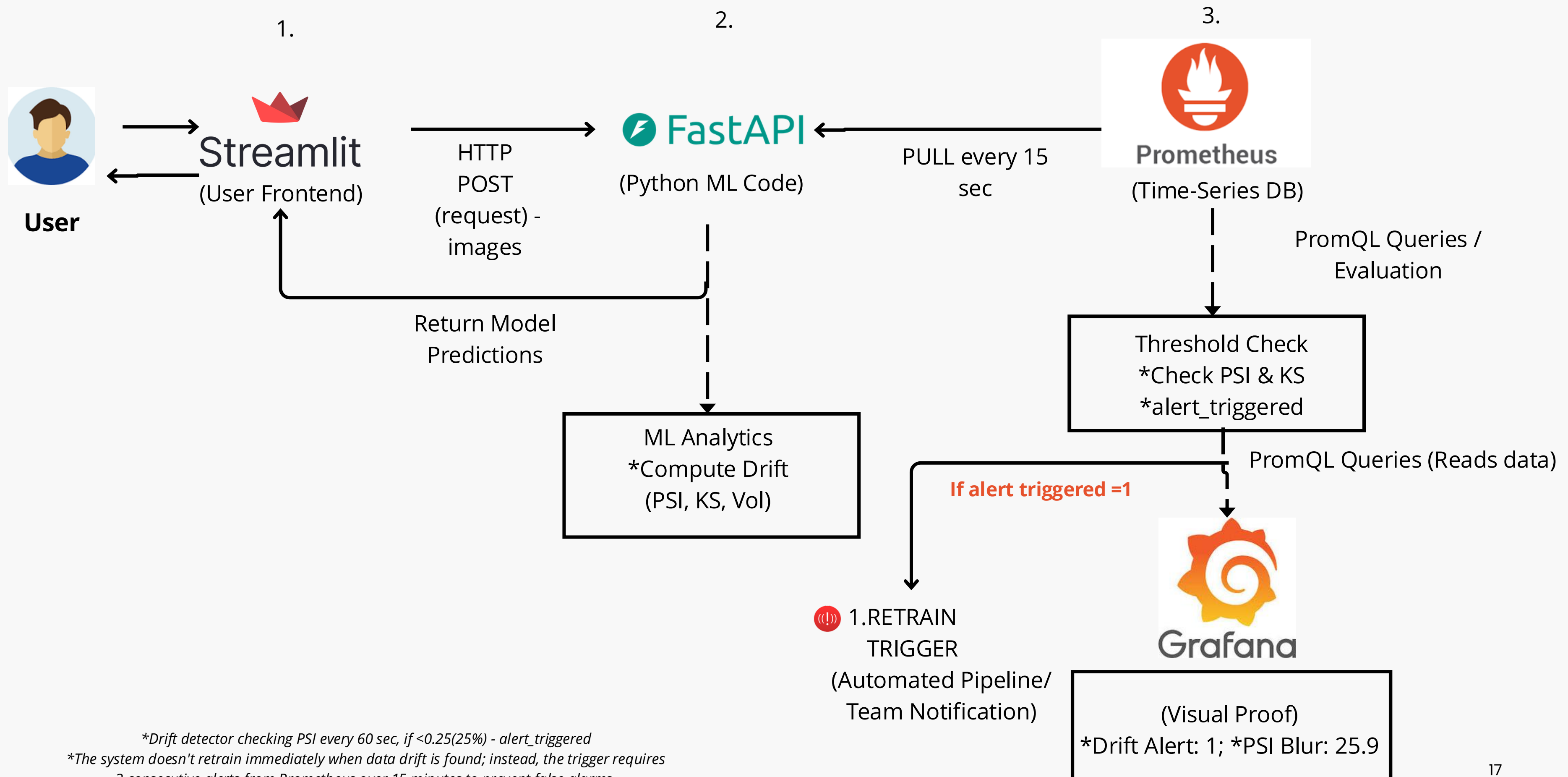
Wait for API health

```
1  ▶ Run for i in {1..30}; do
17  curl: (56) Recv failure: Connection reset by peer
18  Waiting for API... (1/30)
19  curl: (52) Empty reply from server
20  Waiting for API... (2/30)
21  API healthy
22  {"status":"ok","model_loaded":true,"model":"dav2_vitb","device":"cpu"}
```

- > ✓ Smoke /predict/json and /metrics

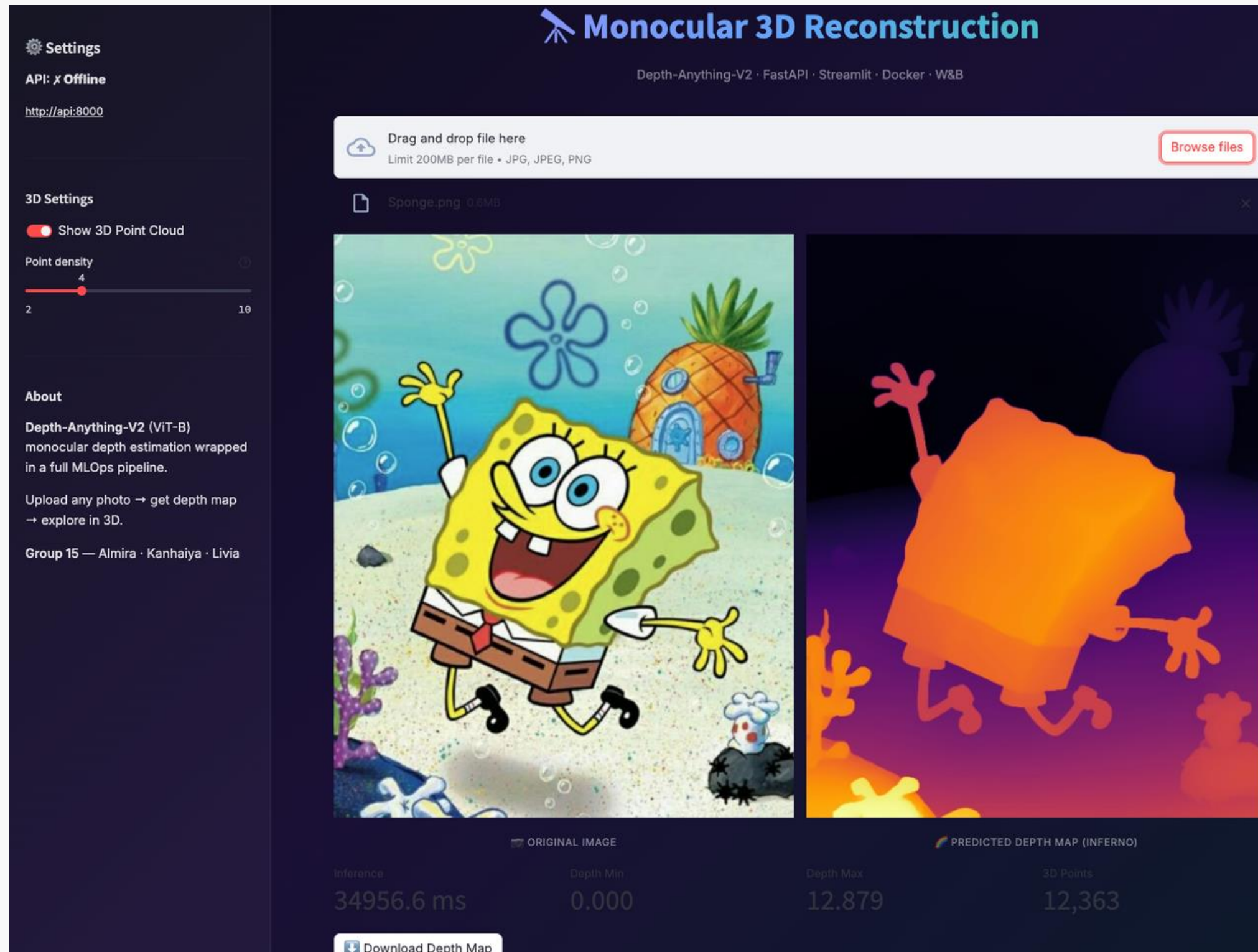
```
43  /predict/json response keys OK
44  /metrics smoke OK
```

LOCAL MODEL SERVING & DEPLOYMENT



*Drift detector checking PSI every 60 sec, if $<0.25(25\%)$ - alert_triggered

*The system doesn't retrain immediately when data drift is found; instead, the trigger requires 3 consecutive alerts from Prometheus over 15 minutes to prevent false alarms.



Monocular 3D Reconstruction in Action:

An interactive web application allows users to upload any 2D image and receive a predicted depth map.

- **Real-Time Depth Mapping:** The app outputs an color-coded depth map, where closer objects are brighter (orange/yellow) and background elements are dark (purple/black).
- **In the background:** Streamlit sends an HTTP POST request with the image to FastAPI container, which runs the Depth-Anything-V2 model.
- While the user gets a visual result, FastAPI captures background metadata—like inference time and blur levels—and prepares it for our monitoring layer.



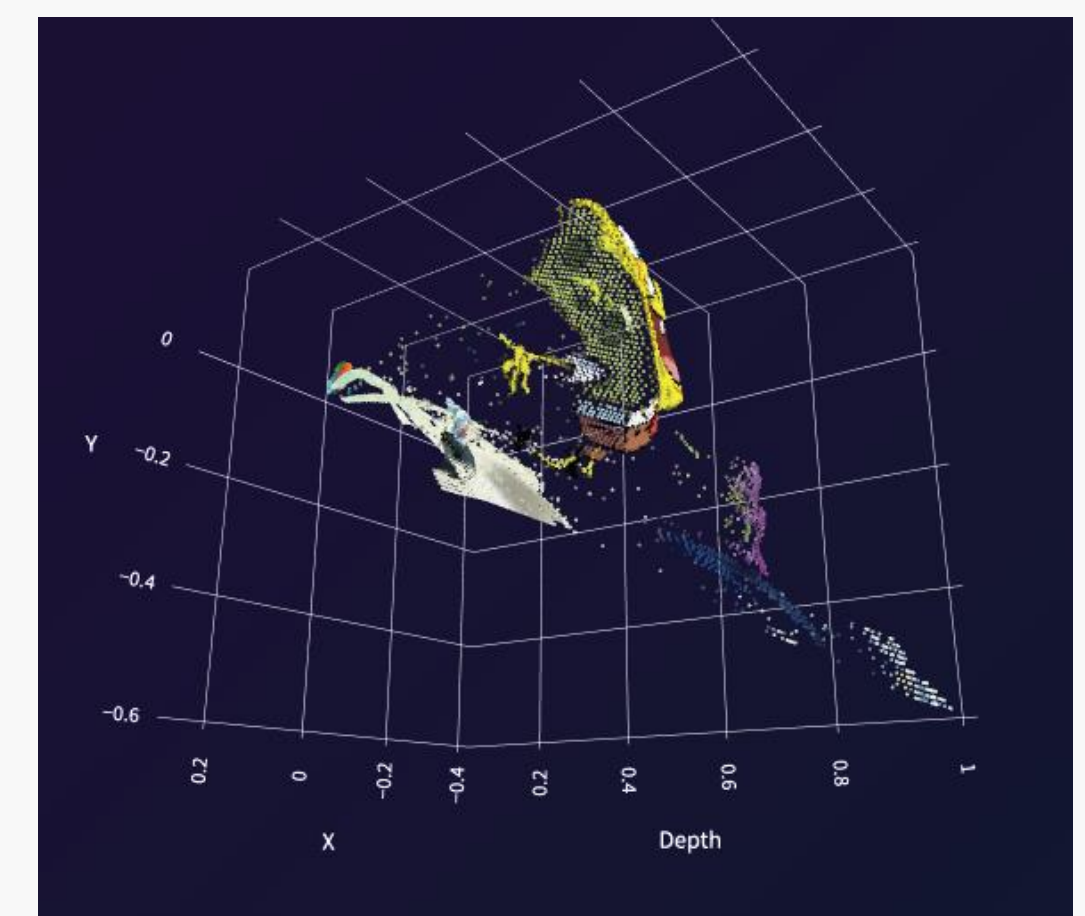
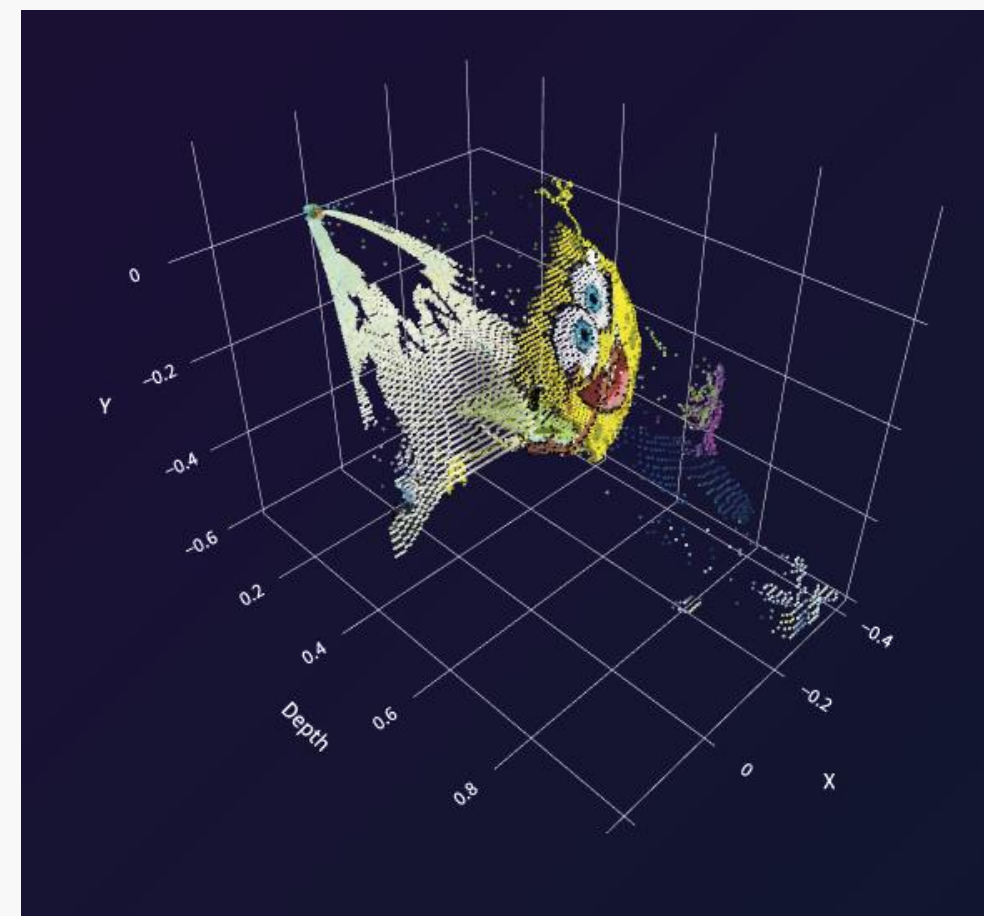
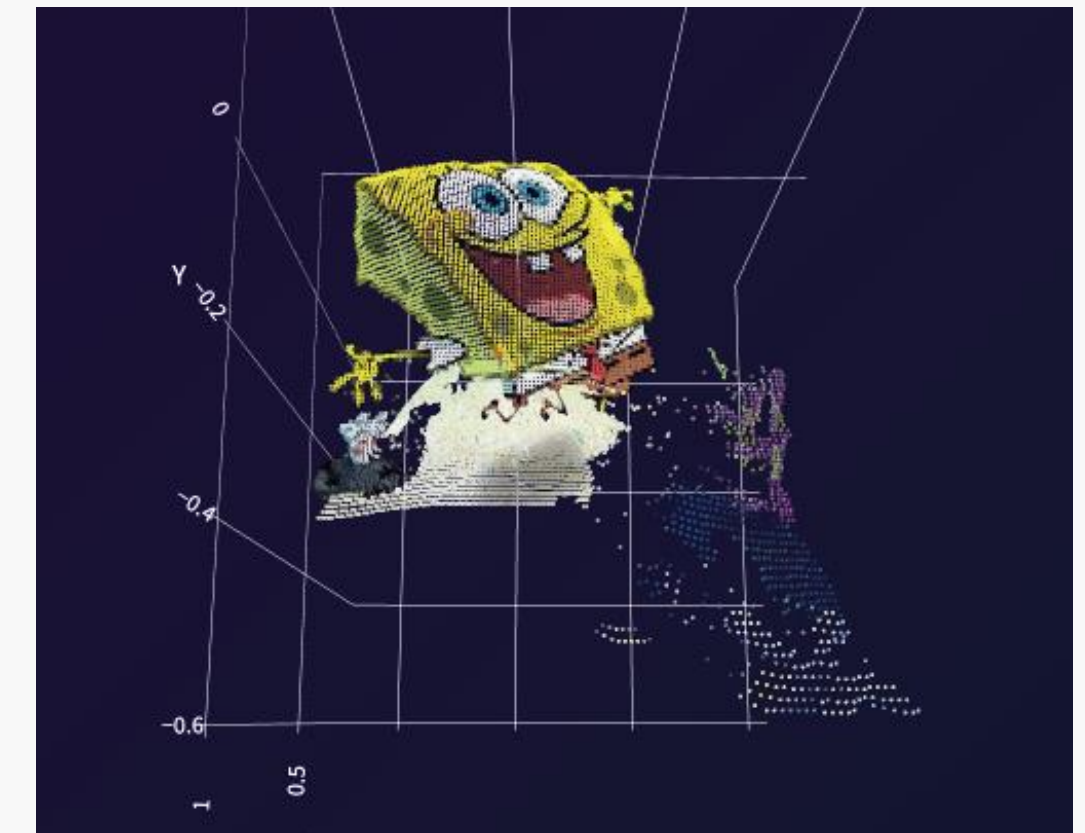
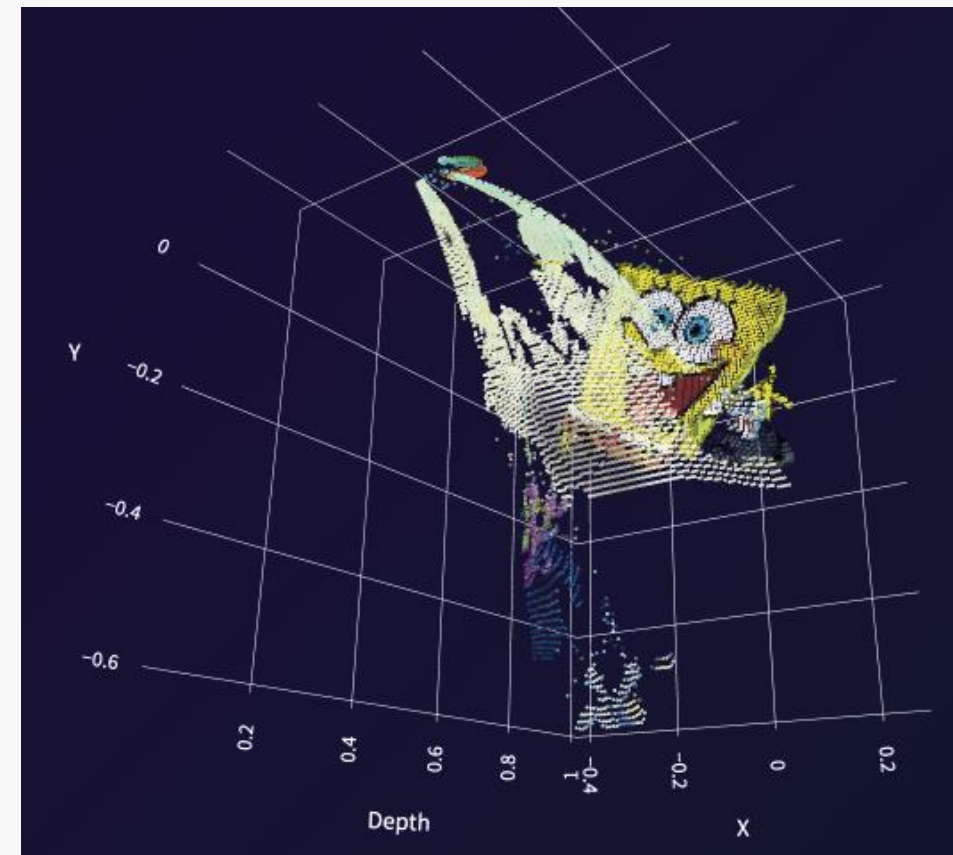
Streamlit

From 2D Image to 3D Geometry: By combining the original RGB pixels with the predicted depth map, the system successfully reconstructs a spatial 3D point cloud, allowing full geometric exploration of the scene.

Technical Breakdown

- Spatial Mapping: Every pixel from the input image is mapped to X, Y, and Z coordinates using the model's depth predictions.
- Interactive Exploration: The frontend allows users to rotate, zoom, and pan the reconstructed object in real time to inspect it from different angles (as shown by the four view perspectives).
- High Density Preservation: The system retains the original colors (RGB) and geometric structures, clearly separating the foreground character from background elements in 3D space.

Interactive 3D point cloud



LOCAL MODEL SERVING & DEPLOYMENT



localhost: 8000

Monocular 3D Reconstruction API 1.0.0 OAS 3.1

[/openapi.json](#)

Depth estimation and 3D point cloud generation using Depth-Anything-V2 (ViT-B). Upload a single RGB image and receive depth map + 3D point cloud data.

default

GET **/metrics** Metrics

Info

GET **/** Root

GET **/health** Health

Monitoring

GET **/drift** Current drift detection report (Stage 5)

Inference

POST **/predict** Returns depth map as colourised PNG image

POST **/predict/json** Returns depth map + point cloud data as JSON

Schemas

The core service is wrapped in an FastAPI backend, cleanly separating model inference, system health tracking, and MLOps monitoring.

Endpoint Structure

- GET **/metrics**: The integration point for Prometheus. It exposes real-time numeric data for scraping.
- GET **/health**: Standard liveness probe to verify if the server container is up and running.
- GET **/drift**: Custom endpoint that computes and returns detailed statistical metrics regarding current data shifts.
- POST **/predict** & **/predict/json**: The primary inference pipelines that process input images and return depth maps

Monitoring

GET

/drift

Current drift detection report (Stage 5)

Returns the latest drift scores computed by the background monitor.

Parameters

No parameters

Execute

Clear

Cancel

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8000/drift' \
  -H 'accept: application/json'
```

Request URL

http://localhost:8000/drift

Server response

Code

Details

200

Response body

```
{
  "psi_brightness": 1.9638,
  "psi_contrast": 13.9166,
  "psi_blur": 25.9256,
  "ks_brightness": 0.5333,
  "depth_mean": 0.3381,
  "depth_std": 0.2122,
  "invalid_ratio": 0.0299,
  "alert": true
}
```

Download

Response headers

```
content-length: 165
content-type: application/json
date: Tue, 26 May 2026 15:28:01 GMT
server: uvicorn
```

Responses

Code

Description

Links

200

Successful Response

No links

Media type

application/json

Controls Accept header.

Example Value

Schema

```
"string"
```

When we test the `/drift` endpoint, it shows us the exact JSON data with our statistical test results. This is the raw truth about our data quality before Prometheus even collects it.

- [PSI \(Blur / Contrast\)](#): Results (25.92 and 13.91) exceed the norm by hundreds of times.
- [KS Test \(Brightness\)](#): Result (0.5333) statistically confirms a drastic shift in image brightness.
- [Invalid Ratio](#): Result (0.0299, about 3%) shows the model predictions are approaching failure limits.

21

LOCAL MODEL SERVING & DEPLOYMENT



Prometheus

localhost: 9090

☐ Use local time ☐ Enable query history ☒ Enable autocomplete ☒ Enable highlighting ☒ Enable linter

Q

{job="fastapi", __name__=~"drift_.*"}

Execute

Table

Graph

Load time: 859ms Resolution: 14s Result series: 8



Evaluation time



drift_input_brightness_psi{instance="api:8000", job="fastapi"}	1.9637507140180028
drift_input_blur_psi{instance="api:8000", job="fastapi"}	25.925554764744305
drift_input_contrast_psi{instance="api:8000", job="fastapi"}	13.916636952772805
drift_pred_depth_mean{instance="api:8000", job="fastapi"}	0.33811724483966826
drift_pred_depth_std{instance="api:8000", job="fastapi"}	0.2121854156255722
drift_pred_invalid_ratio{instance="api:8000", job="fastapi"}	0.029870105173474548
drift_ks_brightness_stat{instance="api:8000", job="fastapi"}	0.5333333333333333
drift_alert_triggered{instance="api:8000", job="fastapi"}	1

Remove Panel

Prometheus acts like an automated inspector, continuously pulling and storing raw technical metrics from our application to track its performance and health in real time.

In the background, Prometheus pulls raw metric data from FastAPI, Grafana visualizes it on dashboards, and if the alert stays triggered, it signals the Retrain Trigger to start a new training cycle

Critical Data Drift Detected: The system successfully identified extreme changes in production data. Drift detector checking PSI every 60 sec, if <0.25(25%) – alert triggered

1. Input Data Drift (Extreme Divergence)

- PSI (Threshold: 0.2): Shows massive distribution shifts across all key image features:
 - Brightness: 1.96 ✖
 - Contrast: 13.92 ✖
 - Blurriness: 25.93 ✖ (Critical)
- KS-Test: 0.53 (Threshold: 0.2) ✖ – Statistically confirms that production images fundamentally differ from the training dataset.

LOCAL MODEL SERVING & DEPLOYMENT



GRAFANA

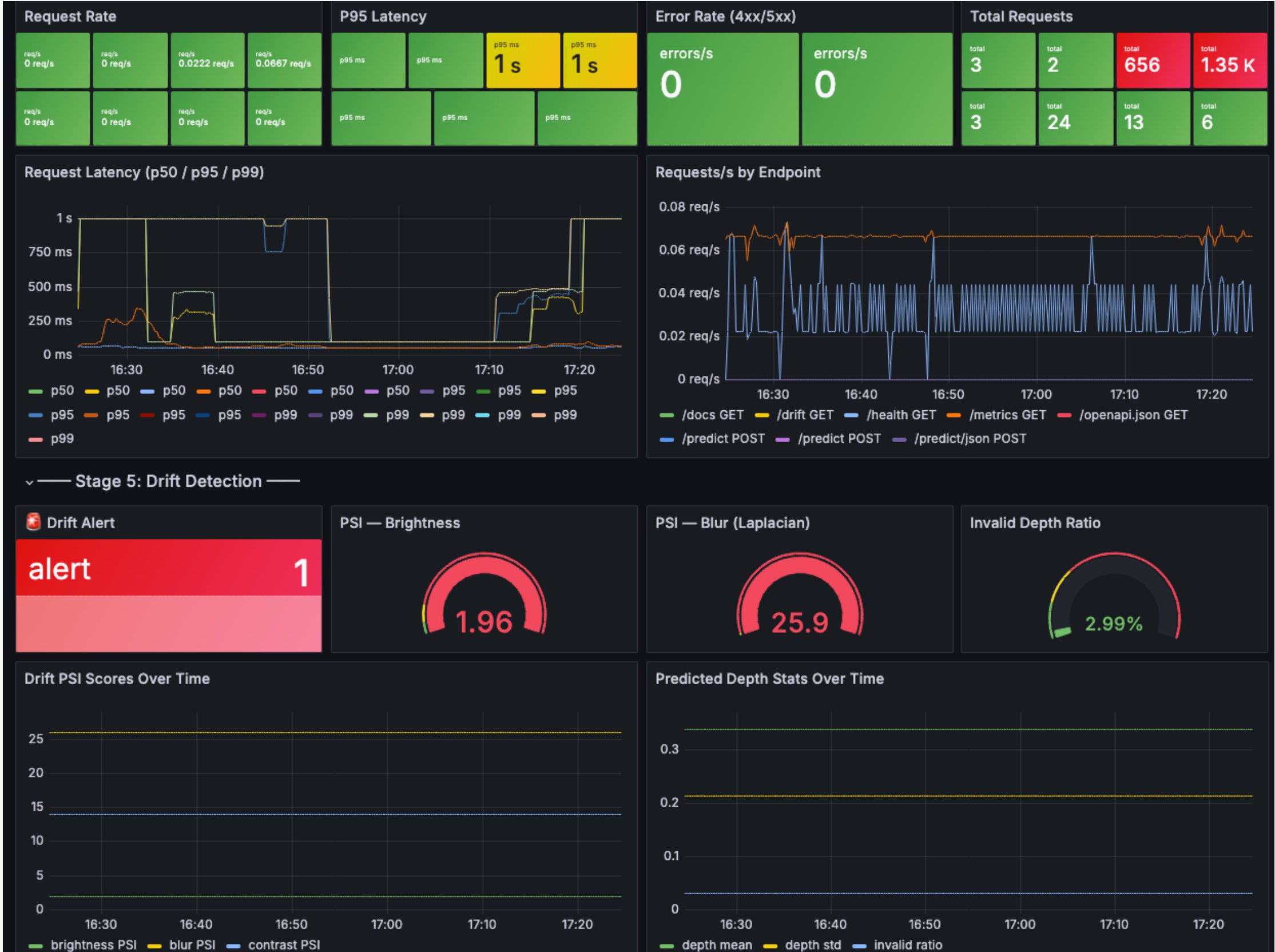
localhost: 3000

Real-time System Degradation Caught:

While infrastructure metrics remain stable with zero errors, the ML monitoring layer successfully caught critical data drift, triggering a high-priority alert.

1. Infrastructure & Performance Metrics

- **Request Rate & Load:** The system handles ongoing traffic, peaking at 1.35 K total requests on the core endpoint.
- **Error Rate:** 0% (zero)— from an engineering standpoint, the API server is completely healthy and working as intended.
- **Latency (p50 / p95 / p99):** System latency mostly stays low, though Grafana captures a few 1-second spikes in P95 latency during specific intervals, which is crucial for infrastructure profiling.
- **Drift Alert:** The system automatically flipped to a red alert state (alert = 1), instantly flagging a critical issue with the data.
- **PSI Brightness & Blur:** Brightness (1.96) and blur (25.9) are way deep into the red zone, meaning the new images are heavily degraded or completely different from the training data.
- **Invalid Depth Ratio:** At 2.99%, the model's failed predictions are right on the edge of our 3% critical warning threshold.



THANK YOU!